JSON

ITI - Assiut Branch

Eng. Sara Ali

JSON

- JSON stands for **J**ava**S**cript **O**bject **N**otation
- JSON is a **text format** for storing and transporting data
- JSON is "self-describing" and easy to understand
- Text-based.
- It can be used instead of XML.
- JSON is used to send data between computers
- JSON is language independent *

Why Use JSON?

- The JSON format is syntactically similar to the code for creating JavaScript objects. Because of this, a JavaScript program can easily convert JSON data into JavaScript objects.

- Since the format is text only, JSON data can easily be sent between computers, and used by any programming language.

- JavaScript has a built in function for converting JSON strings into JavaScript objects:

  JSON.parse()

- JavaScript also has a built in function for converting an object into a JSON string:

  JSON.stringify()

Why Use JSON? (Cont.)

- You can receive pure text from a server and use it as a JavaScript object.
- You can send a JavaScript object to a server in pure text format.
- You can work with data as JavaScript objects, with no complicated parsing and translations.
- **Storing Data**
  - When storing data, the data has to be a certain format, and regardless of where you choose to store it, *text* is always one of the legal formats.
  - JSON makes it possible to store JavaScript objects as text.

JSON Structure

- JSON is built on two structures:

  1. A collection of name:value pairs, separated by (:). Name is string, value is one of JSON values. (Ex.: “Name” : “Ahmed”).

  2 . An ordered list of values. In most languages, this is realized as an array, vector, list, or sequence.

JSON Values Formats

- JSON supports these basic data types:
  - Strings (in double quotes)
  - Numbers (integer or floating point)
  - Booleans (true, false).
  - Objects (in curly braces)
  - Arrays (in square brackets)
  - null

JSON Values Formats (Cont.)

- **String:**
  - Sequence of 0 or more Unicode characters.
  - String values must be wrapped in "double quotes.“.
  - Example: **"Name": "Ahmed“**
- **Number:**
  - Can be: Integer, Real, float, Scientific.
  - Can’t be: octal, hex (Use String instead), NaN or Infinity (Use Null instead).
  - Example:
    - **“rating”: 123**
    - **"length": 122.234**
    - **"Temp": -5**
    - **"atoms per mole": 6.023e+23**

JSON Values Formats (Cont.)

- **Boolean:**
  - Example: **"active": true**
    "active": false
- **null:**
  - Example: **"email": null**
- **Object:**
  - Objects are unordered containers of key/value pairs
  - Objects are wrapped in { }
  - (,) separates key/value pairs
  - (:) separates keys and values
  - Keys are strings
  - Values are JSON values

JSON Values Formats (Cont.)

- **Object (Cont.):**

Example:

```
Var jObj= {
    “id”: 10,
    "company": "ITI" ,
    "city": “Assiut"
}
```

JSON Values Formats (Cont.)

- **Object (Cont.):**
  - JavaScript code Example:

```
<script>
        var sObj= ‘ {“ID”:10,"name": “Ali",“City": “Assiut“, “Job”: null,  “Married”:
true}’;
        var jObj= JSON.parse(sObj);

        //var jobj= {ID:10,name: “Ali",City: “Assiut“, Job: null, Married: true};


        //when embded in JS code, keys aren’t qouted.
        document.write(jobj.ID);  //10
        document.write(jobj[“ID”]); //10
        document.write(jobj.name);  //Ali
        document.write(jobj.Job); //null
</script>
```

JSON Values Formats (Cont.)

- **Array:**
  - Arrays are ordered sequences of values.
  - Arrays are wrapped in [].
  - (,) separates values.
  - Array of string:

    var Arr=["Ahmed", "Ali", "Emad"]
  - Array of Numbers:

    var Arr =[30, 20, 10]
  - Array of different Data Types:

    var Arr =["Ahmed", 10, null, true]

JSON Values Formats (Cont.)

- **Array (cont.):**
  - Array of Objects:

```
var employees=  [
    {firstName: "John", lastName: "Doe"},
    {firstName: "Anna", lastName: "Smith"},
    {firstName: "Peter", lastName: "Jones"}
]
```

JSON Values Formats (Cont.)

- **Array (cont.):**
  - **Example**

```
<script>
        //Array of one type
         var stdArr=["Ahmed","Ali","Emad"];
         for (i=0;i<3; i++)
         {
            document.write(stdArr[i]);
         }

        //Array with different types
        var Jarr=["red", 1, null, true];
        document.write(Jarr[2]);

        //Nested array
        var Jarr2=["red",1,["a", "b", "c"],false];
        document.write(Jarr2[2][1]);

        //Object in an array
        var Jarr3=["Ahmed", {Grade:"A", Mark:20},2];
        document.write(Jarr 3[1].Grade);
</script>
```

JSON vs. XML

- Both JSON and XML can be used to receive data from a web server.
- The following JSON and XML examples both define an employees object, with an array of 3 employees:

```
{"employees":[
  { "firstName":"John", "lastName“:"Doe“},
{"firstName":"Anna", "lastName“:"Smith" },
  { "firstName":"Peter", "lastName":"Jone}
]}
```

```
<employees>
  <employee>
    <firstName>John</firstName> <
lastName>Doe</lastName>
  </employee>
  <employee>
    <firstName>Anna</firstName> <
lastName>Smith</lastName>
  </employee>
  <employee>
    <firstName>Peter</firstName>
<lastName>Jones</lastName>
  </employee>
</employees>
```

JSON vs. XML (Cont.)

- Much Like XML:
  - Both JSON and XML is plain text
  - Both JSON and XML is "self-describing" (human readable)
  - Both JSON and XML can be parsed and used by lots of programming languages
  - Both JSON and XML can be fetched with an HttpRequest
- Much Unlike XML:
  - JSON doesn't use tags
  - JSON is shorter
  - JSON is quicker to read and write
  - JSON can use arrays
- The biggest difference:

XML has to be parsed with an XML parser, JSON can be parsed by a standard JavaScript function.

JSON vs. XML (Cont.)

- **JSON supports these basic data types:**

  Strings, Numbers, Booleans (true, false), Objects, Arrays, null

- **JSON Has No Validator:**

  Ultimately, every application is responsible for validating its inputs.

- **JSON Is Versionless:**

  JSON is very stable. No revisions to the JSON grammar are anticipated.

- **JSON Is Not XML**
  - JSON(Object, Array, String, number, Null).
  - XML(element, Attribute, attribute string, Content, Entities, Declarations, Schema, Version, namespace).

Why JSON is Better Than XML

- **Lighter and faster than XML as on the wire data format.**
- **JSON objects are typed while XML data is typeless.**

  - **JSON types:** string, number, array, boolean, object.

  - **XML data:** are all string

- **Native data form for JavaScript code**

  - XML is much more difficult to parse than JSON.
  JSON is parsed into a ready-to-use JavaScript object.

  - Retrieving values is as easy as reading from an object property in your JavaScript code

Why JSON is Better Than XML (Cont.)

- **For AJAX applications, JSON is faster and easier than XML:**
- **Using XML**:
  - Fetch an XML document.
  - Use the XML DOM to loop through the document.
  - Extract values and store in variables.

- **Using JSON**:
  - Fetch a JSON string.
  - JSON.Parse the JSON string.

How to read JSON files from the server?

- The file type for JSON files is ".json" (or even .txt files), it’s just text files.
- **JSON Http Request**: A common use of JSON is to read data from a web server, and display the data in a web page is using Http Request (XMLHttpRequest Object).
- Http Request is also a technique used in Ajax.

JSON.stringify()

- A common use of JSON is to exchange data to/from a web server.
- When sending data to a web server, the data has to be a string.
- Convert a JavaScript object into a string with JSON.stringify().
- Imagine we have this object in JavaScript:

```
const obj = {name: "John", age: 30, city: "New York"};
```

- Use the JavaScript function JSON.stringify() to convert it into a string.

```
const myJSON = JSON.stringify(obj);
//'{"name":"John","age":30,"city":"New York"}'
```

- myJSON is now a string, and ready to be sent to a server:

JavaScript

THANK YOU